03 / EXERCISE GUIDE

EX29 — 2D rotations and matrices

Class 18 · Mathematics first, physical application second

Goal and scope
Build a 2×2 rotation matrix, use it to rotate vectors, verify algebraic results and interpret a force expressed in different frames. Only one Python file is needed. No JSON, `pytest`, classes or data-file input.

1. Exercise location

```
Python_Engineering/
  02_numerical_methods/
    algebra/
      class_18_matrices/
        EX28_matrix_operations.py
        EX29_rotation_2D.py       <-- NEW
        docs/
          ES/
          EN/
```

Working file: `EX29_rotation_2D.py`. Import NumPy with `import numpy as np`.

2. First concept to implement: $R(\theta)$

Before rotating any vector, define the angle θ and construct its general rotation matrix:

$$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

This is a general expression, valid for any angle. In Python the matrix is not symbolic: choose a numerical value of θ, compute its sine and cosine, and assemble the numerical matrix. Whenever the angle changes, calculate sine, cosine and the matrix again (for now, without defining a custom function).

1. Choose `theta_deg`, for example 30°; convert it to `theta_rad`.
2. Calculate sine and cosine of `theta_rad`.
3. Construct R as a 2×2 array, with correct signs and positions.
4. Multiply Rv using `@` and check the result.
5. Change `theta_deg` and rebuild R to repeat the experiment.

Learning idea: rather than storing several precomputed matrices, the goal is to build $R(\theta)$ for any required angle. Later this process will be encapsulated in a reusable function.

3. Model assumptions and notation

All vectors are mathematical **column vectors** with two components, represented as NumPy arrays of shape `(2,)`. We use a right-handed Cartesian plane and positive counterclockwise angles. Part A uses **dimensionless** vectors; Part B uses forces in newtons.

4. Part A0 — Guided algebraic example

Let $\mathbf{v} = (2, 1)^T$, with a $+90°$ counterclockwise rotation. Work it out manually:

$$R(90°) = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}, \qquad R(90°)\mathbf{v} = \begin{pmatrix} -1 \\ 2 \end{pmatrix}.$$

Reproduce the result with NumPy and verify that the initial and final norms are $\sqrt{5} \approx 2.2361$. This part is meant as a check, not a separate complex problem.

5. Part A1 — Algebraic 30-degree rotation

Inputs: $\mathbf{a} = (4, 0)^T$ and $\theta = +30°$.

1. Convert degrees to radians.
2. Compute sine and cosine, then construct $R(30°)$.
3. Obtain $\mathbf{a}' = R(30°)\mathbf{a}$ using `@`.
4. Calculate $\|\mathbf{a}\|_2$ and $\|\mathbf{a}'\|_2$ and compare.
5. Verify at least one component manually, without Python.

Expected check: $\mathbf{a}' = (2\sqrt{3}, 2)^T \approx (3.4641, 2)^T$.

6. Part A2 — 90-degree rotation and inverse check

Inputs: $\mathbf{q} = (3, -2)^T$ and $\theta = +90°$.

1. Rotate the vector and verify that $\mathbf{q}' = (2, 3)^T$.
2. Apply $R(90°)^T$ to the transformed vector. Is $\mathbf{q}$ recovered?
3. Use `np.allclose()` to return one comparison boolean.

7. Part A3 — Composition of two rotations

Inputs: $\mathbf{p} = (2, 0)^T$, first rotation $+30°$, second rotation $-60°$ (both defined relative to fixed axes).

1. Apply the two rotations in sequence.
2. Compare with a single $-30°$ rotation.
3. Check norm preservation and approximate array equality.

Expected check: $\mathbf{p}_{\text{final}} = (\sqrt{3}, -1)^T \approx (1.7321, -1)^T$.

8. Part B — Engineering: rotating a force

A **physical** force of 10 N initially acts along the positive x axis. Its direction rotates $+30°$ while the global axes remain fixed.

$$\mathbf{F}_{\text{initial}} = \begin{pmatrix} 10 \\ 0 \end{pmatrix} \text{ N}.$$

Program the final force vector, its norm, and the magnitude-preservation check. Interpret the signs of the components and display physical units in printed results. **Expected check:** $\mathbf{F}' = (5\sqrt{3}, 5)^T$ N $\approx (8.6603, 5)^T$ N; magnitude 10 N.

9. Part C — Reference-frame change (physical vector unchanged)

A fixed force has **global components** $\mathbf{F}_G = (3, 4)^T$ N. Now the reference **axes** rotate by $+90°$ counterclockwise. The force does not move.

1. Compute its components in the rotated frame, $\mathbf{F}_L = R(90°)^T \mathbf{F}_G$.
2. Recover the global components using $R(90°)\mathbf{F}_L$.
3. Compare the norms and explain in one sentence what changed and what did not.

Expected check: $\mathbf{F}_L = (4, -3)^T$ N, with $\|\mathbf{F}_L\|_2 = 5$ N.

10. Validation checklist

- Dimensions match: R has shape `(2,2)`, each vector shape `(2,)`.
- Do not confuse `*` with `@`, or degrees with radians.
- Distinguish *active rotation* $R\mathbf{v}$ from *passive basis change* $R^T\mathbf{v}$.
- Match the mathematical checks while allowing small floating-point differences.

11. Code hints: placed at the end

These are **tools**, not the exercise solution. The programmer must choose the algorithm and variable names.

```python
import numpy as np

angle_rad = np.deg2rad(30.0)  # degrees -> radians
c = np.cos(angle_rad)         # scalar cosine
s = np.sin(angle_rad)         # scalar sine

# Matrix-vector product: R @ vector
# Inverse rotation for an orthogonal R: R.T @ rotated
# Approximate array comparison: np.allclose(result, expected)
```